

Московский авиационный институт
(национальный исследовательский университет)

Факультет информационных технологий и прикладной
математики

Кафедра вычислительной математики и программирования

Лабораторная работа №4 по курсу «Дискретный анализ»

Студент: Д. О. Кочев
Преподаватель: А. А. Кухтичев
Группа: М8О-206Б-22
Дата:
Оценка:
Подпись:

Москва, 2024

Лабораторная работа №4

Задача: Необходимо реализовать один из стандартных алгоритмов поиска образцов для указанного алфавита.

Вариант алгоритма: Поиск одного образца при помощи алгоритма Апостолико-Джанкарло.

Вариант алфавита: Слова не более 16 знаков латинского алфавита (регистронезависимые).

Запрещается реализовывать алгоритмы на алфавитах меньшей размерности, чем указано в задании.

1 Описание

В задаче требуется реализовать алгоритм Апостолико-Джанкарло. Основная его идея заключается в том, чтобы завести вспомогательный массив M и при этом в точности воспроизводить сдвиги алгоритма Бойера-Мура.

В ячейке $M[j]$ массива M содержится число, которое меньше или равно числу символов после $(j+1)$ -го элемента текста, которые совпадают с максимальным суффиксом искомой строки. Далее при сравнении элементов, мы сначала обращаемся к этому массиву, чтобы подкорректировать наши действия и уменьшить количество сравнений.

Пользуясь теоремой из [1]: «Модифицированный алгоритм Апостолико-Джанкарло выполняет не более $2m$ сравнений символов и не более $\Theta(m)$ дополнительных действий», мы убеждаемся в скорости и эффективности данного решения.

2 Исходный код

Сначала я реализовал Z-функцию, так как с ее помощью вычисляются все остальные вспомогательные функции. Алгоритм Апостолико-Джанкарло я реализовал с помощью цикла *while* внутри функции *main*. Внутри цикла происходит проверка свойств *M*-массива, исходя из которых, алгоритм будет делать сдвиг или сравнивать дальше, а также заполнять массив *M*. Вспомогательные функции я реализую напрямую в функции *main*.

```
1 std::vector<int> M(text.size(), 0);
2 // N-function.....
3 std::vector<std::string> reversed_pattern = pattern;
4 reverseVector(reversed_pattern);
5 std::vector<int> N = zFunction(reversed_pattern);
6 reverseVector(N);
7 //.....
8
9 // L-function.....
10 std::vector<int> L(pattern_length, 0);
11 for (int i = 0; i < pattern_length; i++){
12     if (N[i] != 0) {
13         L[pattern_length - N[i]] = i + 1;
14     }
15 }
16 //.....
```

laba4.cpp	
std::vector<std::pair<int,int>> processStrings(const std::vector<std::string>& string_vector)	Создание вектора пар для правильного вывода
void reverseString(std::string& s)	Функция для разворота строки
template <typename T> void reverseVector(std::vector<T>& v)	Функция для разворота вектора
std::vector<int> zFunction(const std::vector<std::string>& s)	Возвращает Z-функцию для строки
int UPPS(std::string letter, int index)	Возвращает сдвиг по усиленному правилу плохого символа.

3 Консоль

```
dmitrykochev$ cat test01.txt
a      a
a      a

a      a

a
a
dmitrykochev$ ./laba4 <test01.txt
1,1
1,2
3,1
3,2
6,1
dmitrykochev$ cat test02.txt
cat dog cat dog bird
CAT dog
CaT Dog Cat DOG bird
CAT dog cat dog bird
dmitrykochev$ ./laba4 <test02.txt
2,1
3,1
```

4 Тест производительности

Тест производительности представляет из себя следующее: одни и те же паттерн и текст мы будем использовать как в алгоритме Апостолико-Джанкарло, так и в наивном алгоритме поиска подстроки в строке. Тестов будет три, тест характеризуется тремя числами: количество слов в паттерне, количество слов в тексте, примерный процент вхождения паттерна в строку. Первый тест будет содержать 10^3 слов в паттерне, 10^5 слов в тексте. Второй 10^3 слов в паттерне, 10^6 слов в тексте. Третий 10^4 слов, 10^7 слов соответственно. Длина слова не более 16 символов. Паттерн в тексте будет встречаться не менее, чем в 30% случаев.

```
dmitrykochev$ ./benchmark <test1.txt
Apostolico-Giancarlo algorithm time: 4204us
native algorithm time: 3600us
dmitrykochev$ ./benchmark <test2.txt
Apostolico-Giancarlo algorithm time: 182461us
native algorithm time: 224663us
dmitrykochev$ ./benchmark <test3.txt
Apostolico-Giancarlo algorithm time: 200395us
native algorithm time: 237488us
```

Как можно заметить, при увеличении текста алгоритм Апостолико-Джанкарло становится заметно быстрее наивного алгоритма.

Действительно, как говорилось выше, поиск подстроки в строке в алгоритме Апостолико-Джанкарло выполняется за $\Theta(m)$, в то же время в наивном алгоритме за $\Theta(m * n)$, где m - длина текста, n - длина паттерна. Так как оба алгоритма предназначены для сравнения слов, а не символов, по-настоящему большие тесты сделать не получается, так как в каждом слове не более 16 символов, и обработка таких больших файлов в нужный алгоритму вид занимает очень много времени. Но даже на тестах среднего размера мы видим преимущество алгоритма Апостолико-Джанкарло. Это соответствует расчетам.

5 Выводы

Выполнив первую лабораторную работу по курсу «Дискретный анализ», я научился писать и тестировать алгоритм Апостолико-Джанкарло. Безусловно, это дало мне ценный опыт в написании алгоритмов поиска подстроки своими руками с нуля. Ввиду особенностей данного алгоритма, мне также приходилось разбираться с алгоритмами Бойера-Мура, КМП, и осознать их. Во время выполнения я сталкивался с большим количеством проблем, так как алгоритм довольно сложен для осознания. В итоге я был удивлен, насколько алгоритм Апостолико-Джанкарло - изящное решение задачи поиска подстроки в строке. Это невероятно ценные и важные знания. Они очень помогут мне в будущей разработке сложных задач и программ.

Список литературы

- [1] Dan Gusfield, *Algorithms on string, trees, and sequences*, Cambridge University, 1997.